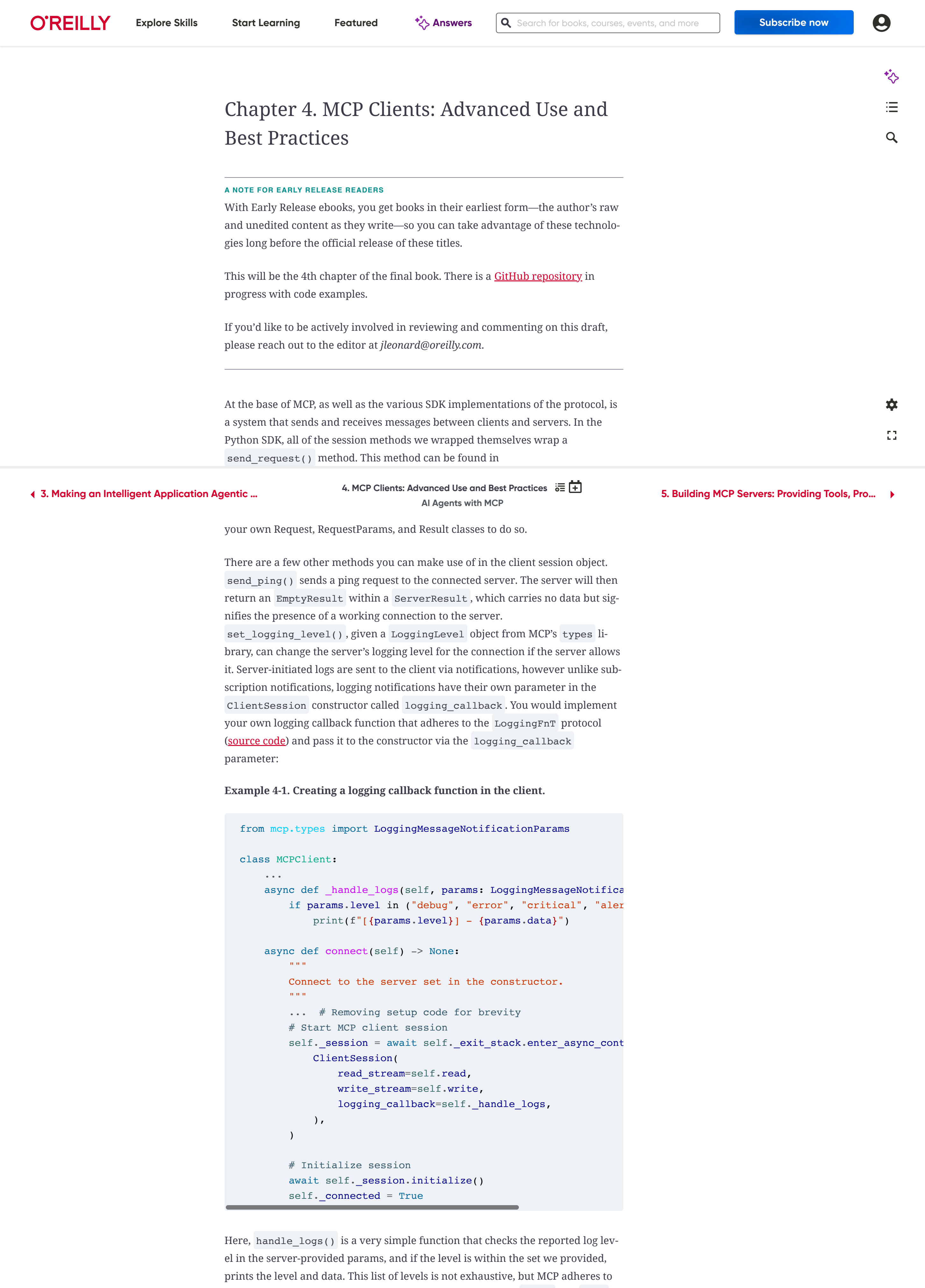

Chapter 4. MCP Clients: Advanced Use and Best Practices

A NOTE FOR EARLY RELEASE READERS

With Early Release ebooks, you get books in their earliest form—the author's raw and unedited content as they write—so you can take advantage of these technologies long before the official release of these titles.

This will be the 4th chapter of the final book. There is a GitHub repository in progress with code examples.

If you'd like to be actively involved in reviewing and commenting on this draft, please reach out to the editor at *jleonard@oreilly.com*.

At the base of MCP, as well as the various SDK implementations of the protocol, is a system that sends and receives messages between clients and servers. In the Python SDK, all of the session methods we wrapped themselves wrap a `send_request()` method. This method can be found in

your own Request, RequestParams, and Result classes to do so.

There are a few other methods you can make use of in the client session object. `send_ping()` sends a ping request to the connected server. The server will then return an `EmptyResult` within a `ServerResult`, which carries no data but signifies the presence of a working connection to the server. `set_logging_level()`, given a `LoggingLevel` object from MCP's `types` library, can change the server's logging level for the connection if the server allows it. Server-initiated logs are sent to the client via notifications, however unlike subscription notifications, logging notifications have their own parameter in the `ClientSession` constructor called `logging_callback`. You would implement your own logging callback function that adheres to the `LoggingFnT` protocol (source code) and pass it to the constructor via the `logging_callback` parameter:

Example 4-1. Creating a logging callback function in the client.

```
from mcp.types import LoggingMessageNotificationParams

class MCPClient:
    ...
    async def _handle_logs(self, params: LoggingMessageNotifica
        if params.level in ("debug", "error", "critical", "aler
            print(f"[{params.level}] - {params.data}")

    async def connect(self) -> None:
        """
        Connect to the server set in the constructor.
        """
        ...  # Removing setup code for brevity
        # Start MCP client session
        self._session = await self._exit_stack.enter_async_cont
            ClientSession(
                read_stream=self.read,
                write_stream=self.write,
                logging_callback=self._handle_logs,
            ),
        )

        # Initialize session
        await self._session.initialize()
        self._connected = True
```

Here, `handle_logs()` is a very simple function that checks the reported log level in the server-provided params, and if the level is within the set we provided, prints the level and data. This list of levels is not exhaustive, but MCP adheres to the syslog severity levels defined in RFC 5424 . In addition to `level` and `data`, `LoggingMessageNotificationsParams` also provides an optional `logger` name and `model_config` dictionary.

`send_progress_notification()` sends a notification to the server with some `progress_token` and the amount of `progress` as a float. Optionally, you can also send a `total` float and `message` string to the server. How these values end up looking will depend on the server you're connecting to and what the server requires for progress updates.

Providing MCP Client Capabilities

In the previous section, you saw an example of implementing a callback to handle log messages from a connected MCP server. In this one, you'll see this pattern repeated, but in service of providing client capabilities to the connected MCP server. There are three major capabilities that a client can provide to a server: **sampling**, where the server is able to request chat completions from the LLM the client is connected to via the client, **roots**, which indicate to servers what areas in the host application's filesystem the server has access to, and **elicitations** which allow the connected server to request information from the user.

Servers are not required to use or respect these capabilities, but if your client provides them, you are required to advertise that your client does provide them, which happens during the initialization phase of the client-server connection. If you are using the Python SDK, this occurs automatically when you implement a callback function for the capability or capabilities that you are supporting and pass it to the `ClientSession` constructor via the appropriate parameter. For providing client capabilities, the general workflow is as follows:

1. In your client class, implement a callback function that will be called when the server requests it.
2. Ensure that the callback follows the established protocol for the capability, such as `SamplingFnT`.
3. Pass the callback function to the `ClientSession` constructor via the appropriate parameter, such as `sampling_callback` for sampling.

And that's it. Of course, the majority of the effort comes in step 1, and will be unique depending on your use case, interaction flow.

Supporting Sampling

Sampling is a client capability that allows MCP server to make use of whichever LLM the client and its host application is connected to. This opens up agentic workflows from the MCP server, such as allowing nested tool choice and use, or for the server to ask the LLM a question and use that to choose or fill information into a prompt that it provides, or whatever else your imagination can come up with.

At the protocol level, this begins with the server sending a `sampling/createMessage` request to the client to initiate sampling. When the client receives this request, it should trigger a human-in-the-loop (HITL) workflow to get the user's approval. Once the user has approved the request, the client can then forward the message to the LLM for a response. The response comes back to the client, which presents another opportunity for user approval before returning the response to the server. Once the response is approved, the client can return the response to the server.

Because this is a client-provided capability, there is slightly less work to do here than there is for using server-provided capabilities. Like you saw with logging, the `ClientSession` class provides a `SamplingFnT` protocol which defines the signature of the sampling callback function you will implement, and the constructor has a `sampling_callback` parameter that will take your callback to be used if and when a connected server sends a sampling request. When the client receives a sampling request, it will call your callback function with a `context` parameter of type `RequestContext` and a `params` parameter of type `CreateMessageRequestParams`, and expect either a `CreateMessageResult` or an `ErrorData` object to be returned. The `CreateMessageRequestParams` object contains the parameters for the sampling request:

- `messages`: a list of `SamplingMessage` objects, which holds the properties `role` and `content`, which you will use to construct a message to send to your LLM.
- `modelPreferences`: an optional parameter which gives the client a hint as to

the preferred model to use for the sampling request along with several priority preferences (cost, speed, and intelligence). It is up to you to decide whether to respect this preference or not.

- `systemPrompt`: also optional, this holds a server-provided system prompt that you can either use with your existing system prompt, on its own (overriding your own system prompt), or not at all.
- `includeContext`: this is an optional string literal that determines whether the server wants to include the context of a request in the response, either from the connected server or all servers.
- `temperature`: an optional float to control the temperature of the LLM's generation.
- `maxTokens`: an optional integer to control the maximum number of tokens to use for generation.
- `stopSequences`: an optional list of strings that signify the end of a generation.

The `CreateMessageResult` object that your callback will return is simpler: it includes the `role` of the message, its `content` (which can be `TextContent`, `ImageContent`, or `AudioContent`), a string `model` representing the model that was used to generate the response, and a `stopReason` that indicates why the generation stopped.

WARNING

Because of the risk of giving an external server access to the LLM your application is using and that you are paying for, both myself and Anthropic strongly recommend that you implement some form of human-in-the-loop (HITL) approval process before a server is allowed to send a request to an LLM via your client and application.

In this example, we will follow the general workflow above, by creating a simple sampling callback that implements the `SamplingFnT` interface and passing it into the `ClientSession` constructor. In order to show the basic implementation, we will refrain from implementing the HITL workflow, since it is highly dependent on the host application and its user interface. In practice, the protocol defines several "shoulds" that you should consider when developing your sampling callbacks:

- You *should* implement a HITL workflow to get user approval before sending a message to the LLM and before returning the response to the server.
- You *should* respect all model preference hints provided by the server.
- You *should* implement rate limiting.

While these aren't required by the protocol, when implementing this in the wild following these guidelines will help you safeguard your users and your LLM from abuse. As such, you will see more complex implementations of sampling callbacks in the later project-based chapters.

Example 4-2. A sampling callback function in the client.

```python
# client.py
from anthropic import Anthropic
from mcp.shared.context import RequestContext
from mcp.types import (
    CreateMessageRequestParams,
    CreateMessageResult,
    ErrorData,
    SamplingMessage
    TextContent
)

class MCPClient:
    def __init__(self, name: str, server_url: str, llm_client:
        self.name = name
        self.server_url = server_url
        self._session: ClientSession = None
        self.exit_stack = AsyncExitStack()
        self._connected: bool = False
        self._llm_client = llm_client

    async def _handle_sampling(
        self,
        context: RequestContext[ClientSession, None],
        params: CreateMessageRequestParams
    ) -> CreateMessageResult | ErrorData:
        messages = []
        for message in params.messages:
```

```
            if isinstance(message.content, TextContent):
                messages.append(
                    {"role": message.role, "content": message.c
                )
            else:
                # Handle other content types if needed
                messages.append(
                    {"role": message.role, "content": str(messa
                )

        response = self._llm_client.messages.create(
            max_tokens=params.maxTokens,
            messages=messages,
            model="claude-sonnet-4-0",
        )

        # Extract content from the response - content is a list
        if response.content and len(response.content) > 0:
            content = response.content[0]
            if hasattr(content, "text"):
                content_data = TextContent(type="text", text=co
            elif hasattr(content, "data"):
                content_data = BlobResourceContents(
                    type="blob",
                    data=content.data,
                    mimeType=content.mimeType,
                )
            else:
                # Fallback to string representation
                content_data = TextContent(type="text", text=st
        else:
            # No content in response
            content_data = ""

        return CreateMessageResult(
            role=response.role, content=content_data, model="cl
        )

    async def connect(self) -> None:
        """
        Connect to the server set in the constructor.
        """
        ...  # Removing setup code for brevity
        # Start MCP client session
        self._session = await self._exit_stack.enter_async_cont
            ClientSession(
                read_stream=self.read,
                write_stream=self.write,
                logging_callback=self._handle_logs,
                sampling_callback=self._handle_sampling,
            ),
        )

        # Initialize session
        await self._session.initialize()
        self._connected = True
    ...
# agent.py
...
if __name__ == "__main__":
    mcp_client = MCPClient(
        name="calculator_server_connection",
        command="uv",
        server_args=[
            "--directory",
            str(Path(__file__).parent.parent.resolve()),
            "run",
            "calculator_server.py",
        ],
        llm_client=anthropic_client,
    )
    agent = Agent(mcp_client, anthropic_client)
    asyncio.run(agent.run())
...
```

In `agent.py`, we added an `llm_client` parameter to the `MCPClient` constructor, since the client will need access to the LLM client to route messages between the LLM and the server. The handler, `_handle_sampling()` constructs a message from any SamplingMessages sent by the MCP server to the client by extracting each message's `content` and adding it to the `messages` list. The `messages` list is then sent to the LLM for a response via the `_llm_client`. The response is then extracted and transformed into either `TextContent` or

`BlobResourceContents` and sent back to the server in a `CreateMessageResult` object.

WARNING

This implementation is simplified, and not safe for deployment in the real world. Your handler should request permission from the user before sending a server's prompt to the application's model.

In the `connect()` method in `client.py`, we pass the sampling handler method into the `ClientSession` constructor via `sampling_callback`. An MCP server can send many different parameters to the client, all via the `CreateMessageRequestParams` object. This object holds all of the parameters that you can expect to use for creating a sampling message, such as `maxTokens`, which defines the upper bound of tokens to use in a generation, `system_prompt` to define a system prompt, for the LLM call, and `model_preferences` which tells the client which models are best for the server to use. Note that it is not required for the client to respect these preferences. To see all of the parameters that a server is able to send you, you can view the CreateMessageParams class definition in GitHub.

Supporting Roots

Roots are a more recent addition to MCP and are used by clients to indicate which files and directories a connected MCP server has access to. These are usually configured by the user of the host application, like when providing a code assistant a project directory to work from. Because this list can potentially be changed by the user at any time, the client must be able to emit a `ListChanged` notification to the server to let it know that the list of roots has changed. When you support roots with the Python SDK, this is taken care of for you.

WARNING

Roots are not a strong security measure, and should not be used as such. This is because servers can choose to not respect the boundaries set by roots, so your host application should have the final say in which files the server or LLM can actually access.

Beyond supporting `ListChanged` notifications, supporting roots requires that you are be able to send the actual list of roots to the server as well. This gives the server the information it needs to do its filesystem operations. As specified in the protocol, the list is a list of JSON objects that have `uri` and `name` properties. As of this writing, the `uri` property must use the `file://` scheme, and the `name` property can be any string that provides a human-readable name for the root. In the Python SDK, roots are represented by the `Root` class, which has the `uri` and `name` properties, but also a `_meta` property. This is a dictionary that can be used to store any additional metadata about the root where keys have an optional dot-separated prefix that ends with a `/` and a required name, and the value can be whatever you want.

To support roots in your client using the Python SDK, you will implement a callback function and pass it to the session constructor just like you did for logging and sampling, using the parameter `list_roots_callback`. The callback function has to fulfill the `ListRootsFnT` protocol, which takes a `context` parameter of type `RequestContext`, specifically a `ClientSession` context, and returns either a `ListRootsResult` or an `ErrorData` object. `ListRootsResult` is a subclass of the `Result` class, and has a `roots` property that is just a list of `Root` objects. It is up to you to decide how your host application gathers the list of roots from your users and passes it to the client, however. The `ErrorData` class allows you to handle JSON-RPC error data, and has the properties `code` for the JSON-RPC error code, `message` for the error message, and `data` for the error data.

Because of the security implications of providing explicit access to the host machine's filesystem, the protocol provides several "musts" and "shoulds" that you either must or should follow when implementing roots support in your client. These are:

- You *must* only expose roots with permissions appropriate for the intended use.
- You *must* validate root URIs to avoid path traversal attacks.
- You *must* implement access controls to further prevent unauthorized access to files and directories.
- You *must* monitor the accessibility of roots to ensure they are not blocked by

firewalls or other network restrictions.

- You *should* get user consent before providing roots to the server.
- You *should* provide an understandable user interface for adding, removing, and inspecting roots and their availability.

You can find the full and up-to-date list of security and implementation guidelines in the MCP specification itself.

In the example below, you will see a simple `_handle_roots()` callback function that returns a list of `Root` objects for the server to use. This approach sets up an instance variable in the `MCPClient` class to store any roots that the host application provides, which the callback can then access. This also allows the list of roots to be updated by the host application at any time.

Example 4-3. Implementing a callback to support roots.

```
# client.py
from mcp.types import ListRoolsResult, Root

class MCPClient:
    def __init__(
        self,
        name: str,
        command: str,
        server_args: list[str],
        llm_client: Anthropic,
        env_vars: dict[str, str] = None,
        file_roots: list[str] = None,
    ) -> None:
        self.name = name
        self.command = command
        self.server_args = server_args
        self.env_vars = env_vars
        self.file_roots = file_roots
        self._session: ClientSession = None
        self._exit_stack: AsyncExitStack = AsyncExitStack()
        self._connected: bool = False
        self._llm_client = llm_client
    ...
    async def _handle_roots(
        self,
        context: RequestContext[ClientSession, Any],
    ) -> ListRootsResult | ErrorData:
        """
        Roots handler that returns the file roots, implementing
        """
        roots_result = []
        for root in self.file_roots:
            if not root.startswith("file:///"):
                logger.warning(f"Root {root} does not start wit
            else:
                roots_result.append(Root(uri=root))
        if roots_result is None:
            return ErrorData(code=-32602, message="No valid fil
        return ListRootsResult(roots=roots_result)

    async def connect(self) -> None:
        ...
                # Start MCP client session
        self._session = await self._exit_stack.enter_async_cont
            ClientSession(
                read_stream=self.read,
                write_stream=self.write,
                logging_callback=self._handle_logs,
                sampling_callback=self._handle_sampling,
                list_roots_callback=self._handle_roots,
            ),
        )
        ...
# agent.py
...
if __name__ == "__main__":
    mcp_client = MCPClient(
        name="calculator_server_connection",
        command="uv",
        server_args=[
            "--directory",
            str(Path(__file__).parent.parent.resolve()),
            "run",
            "calculator_server.py",
        ],
        llm_client=anthropic_client,
```

```
        file_roots=[
            f"file:///{str(Path(__file__).parent.resolve())}",
        ],
    )
    agent = Agent(mcp_client, anthropic_client)
    asyncio.run(agent.run())
```

In this example, we've made some small and some bigger changes to the client. The major change was adding the `_handle_roots()` callback function to the client class. The function is called when the server sends a `ListRoots` request, and it returns a list of `Root` objects for the server to use. While it's entirely up to you to decide how to gather and handle the list of roots from your host application or users, this code just checks a new instance variable `file_roots` to see if any roots were passed in to the client constructor. This approach also allows the host application to update the list of roots at any time, which gives this technique a lot of flexibility.

The `_handle_roots()` function does some minor checks like ensuring each root begins with the `file://` scheme and omits the fully qualified hostname of the machine, then creates a `Root` object from the root URI string. If there are no roots, it returns an `ErrorData` object with a JSON-RPC error code of `-32602` and a message indicating that no valid file roots were provided. If there are roots, those are returned in a `ListRootsResult` object. The `ClientSession` constructor was also updated to pass in the `_handle_roots()` callback function to the `list_roots_callback` parameter.

In `agent.py`, the changes are minimal: we only need to pass the `file_roots` list to the `MCPClient` constructor, which will then be used by the client in the `_handle_roots()` function. If you're following along in the GitHub repository, try out this version of our MCP-powered agent by asking it to tell you how many files are in a directory. Ask it things like "how many files are in `/full/path/to/directory`?" for a directory that's within your given root boundaries, and again for one outside of those boundaries. An actual conversation using the example code (including logging statements) is shown below, where the server is restricted by the roots provided by the client:

Example 4-4. Output from the roots example.

```
You: How many files are in /full/path/to/ai_agents_mcp_examples

Using tool: count_files
Processing request of type CallToolRequest
[error] - Access denied: /full/path/to//ai_agents_mcp_examples/

Assistant: It looks like I don't have access to that specific d

I can only count files within the allowed directory. Would you
```

The next and final client-provided capability is the ability to support **elicitations**, or human feedback.

Supporting Elicitations

Elicitations are a newer feature in the Model Context Protocol that allows MCP servers to request additional information from users via the client. A common use for elicitations is to add a human to the agent loop to help it complete tasks that require user input, such as getting a clarification on a task, or to get a user's approval for a decision. They can also be used to get information from a user as it is needed, rather than forcing the user to provide everything up front in the initial prompt.

Elicitations are server-initiated, and when they're received by the client, the client can respond with one of three types of response types: accept, decline, or cancel. This empowers the application user to control whether they give the requested information to the server or not, as well as how to proceed with the conversation if they decide not to. An accept response means that the user has approved a response to the elicit request and that the response has the requested information. A decline response means that the user has rejected the information request, and a response is returned either without a `content` field or with an empty one. A cancel response is similar to the decline response, but is used to indicate that the user didn't explicitly reject the request. This would be due to the user doing an action that cancels the conversation, such as closing a chat dialog window or modal, without explicitly denying the request.

Servers can also provide a response schema to clients using a subset of JSON, which allows for flat objects that only allow a few property types: strings, numbers, booleans, and enums. This allows for a server to elicit structured feedback from the application user and use it for its own purposes. Luckily, the Python SDK provides Pydantic models for all of the response types and schemas, so that you can use them with a familiar API within your client.

Again, the protocol provides several “shoulds” that you should follow when implementing support for elicitations in your client, as they can be an attack vector for malicious servers gathering personal information from users. These are: - You *should* implement controls that allow a user to accept, decline, or cancel an elicitation request. - You *should* validate elicitation content against the provided schema. - You *should* indicate very clearly to the user which server is requesting the elicitation. - You *should* implement rate limiting. - You *should* present elicitations requests to the user that make it clear what information is being requested and why.

To support elicitations in your client, you will follow a similar pattern to supporting logging, sampling, and roots. You need to implement a callback function that conforms to the `ElicitationFnT` protocol, which takes a `context` parameter that takes a `ClienstSession` request context and a `params` parameter that expects an `ElicitRequesstParams` object. The callback has to return either an `ElicitResult` object or an `ErrorData` object. The `ElicitRequestParams` object comes from the server and is comprised of a string `message` and a `requestedSchema` property of type `ElicitRequestedSchema`, which is a type alias for a Python dictionary. The `ElicitResult` return type consists of a string literal `action` that reflects the user’s response to the request (“accept”, “decline”, or “cancel”) and a `content` dictionary that holds the structured information requested by the server, which is only present if `action` is “accept”.

In the example below, you will see a `_handle_elicitations()` callback implementation. Be sure to notice how it makes very clear that the user is responding to a server-initiated request, and that the client is returning the user’s response to the server. What isn’t shown is the `_collect_form_data()` helper function that parses the elicitation’s request schema and displays it to the user to get structured information. You can find the full code in the GitHub repo for this book.

Example 4-5. Implementing a callback to support elicitations.

```
# source.py
from mcp.types import ElicitResult, ElicitRequestParams, Elicit
...
class MCPClient:
    ...
        async def _handle_elicitation(
        self,
        context: RequestContext[ClientSession, Any],
        params: ElicitRequestParams,
    ) -> ElicitResult | ErrorData:
        """
        Elicitation handler that displays the server request to
        their accept/decline response, and collects form data w
        implementing the ElicitFnT protocol.
        """
        # Get the server name from the client instance
        requesting_server = self.name

        # Display the elicitation request to the user
        print(f"\n{'='*60}")
        print(f"ELICITATION REQUEST FROM SERVER: {requesting_se
        print(f"{'='*60}")
        print(f"Message: {params.message}")
        print(f"{'='*60}")

        # Get user input for accept/decline
        while True:
            user_response = (
                input("\nDo you want to accept this request? (y
                .lower()
                .strip()
            )

            if user_response in ["y", "yes", "accept"]:
                print("Request accepted")
                # Collect form data based on the schema
                form_data = self._collect_form_data(params.requ
                if form_data is not None:
                    print("Form data collected successfully")
```

```
                    return ElicitResult(action="accept", conten
                else:
                    print("Form data collection cancelled")
                    return ElicitResult(action="cancel")
            elif user_response in ["n", "no", "decline"]:
                print("Request declined")
                return ElicitResult(action="decline")
            elif user_response in ["c", "cancel"]:
                print("Request cancelled")
                return ElicitResult(action="cancel")
            else:
                print(
                    "Invalid response. Please enter 'y' (accept
                )
    ...
    async def connect(self) -> None:
        ...
        self._session = await self._exit_stack.enter_async_cont
            ClientSession(
                read_stream=self.read,
                write_stream=self.write,
                logging_callback=self._handle_logs,
                sampling_callback=self._handle_sampling,
                list_roots_callback=self._handle_roots,
                elicitation_callback=self._handle_elicitation,
            ),
        )
        ...
```

As with the other client-provided capabilities, we implement a `_handle*()` callback function and pass it to the `ClientSession` constructor via the `elicitation_callback` parameter. This is how the client is able to advertise its capabilities to the connected server. Within the callback, we use asterisks and text in all caps to make it extremely clear that the server is requesting information from the user, along with the name of the server that is making the request. Then, we start an infinite loop to ask the user to accept, decline, or cancel the request. If the user accepts it, we pass the elicitation's request schema (in `params.requestedSchema`) to the helper function `_collect_form_data()`, which takes the schema, displays it to the user, and gets their input for each property. The result is sent back to the server as an `ElicitResult` object, and the server can do whatever it does with user information. This is written to be as generic as possible: the main feature of MCP is to decouple agentic applications from the tools and resources they use, so the client should need no knowledge of the elicitation or schema that the server is requesting.

Supporting Multiple Models

One of the major benefits of using MCP is that it gives you the freedom, as the application developer, to allow your users to use many models, or the model of your choice. With MCP, you aren't tied to a single model, despite the examples in this chapter working with an Anthropic model. The simplest way to support this would be to write a translation function in your client for each primitive-model family pair that turns MCP objects into the format expected by the model family in question. You can improve on this basic design by creating a class for each primitive that can be constructed from the MCP object and has a translation method as a member of that class. To incorporate this into your client, you could modify `` get_available_tools() to return a list of `InternalTool `` objects to the host application, have the host application call the appropriate translation method on each tool and then send the results to the appropriate LLM.

This workflow is shown in the following example, which starts with a new file called `internal_tool.py` which implements the `InternalTool` class and adds as a member the `translate_to_openai()` method. This just returns a dictionary with the keys `type`, `name`, `description` and `parameters`. The `get_available_tools()` method in the client is updated to return a list of `InternalTool` objects instead of Anthropic API-formatted dictionaries. This now means that the host application can change the model that it is calling without any changes to the client itself, allowing further decoupling of the client and the host application.

Example 4-6. Handling multiple LLMs by allowing the client to translate between them.

```
#internal_tool.py
from typing import Any
```

```
class InternalTool:
    def __init__(
        self, name: str, input_schema: dict[str, Any], descript
    ) -> None:
        self.name = name
        self.input_schema = input_schema
        self.description = description

    def translate_to_openai(self) -> dict[str, Any]:
        return {
            "type": "function",
            "name": self.name,
            "description": self.description,
            "parameters": self.input_schema,
        }

    def translate_to_anthropic(self) -> dict[str, Any]:
        return {
            "name": self.name,
            "description": self.description,
            "input_schema": self.input_schema,
        }


example_tool = InternalTool(name=mcp_tool.name, input_schema=mc
openai_tool = example_tool.translate_to_openai()

# client.py
from internal_tool import InternalTool
...
class MCPClient:
    ...
    async def get_available_tools(self) -> list[dict[str, Any]]
        if not self._connected:
            raise RuntimeError("Client not connected to a serve

        tools_result = await self._session.list_tools()
        if not tools_result.tools:
            logger.warning("No tools found on server")
        available_tools = [
            InternalTool(
                name=tool.name,
                description=tool.description,
                input_schema=tool.inputSchema,
            )
            for tool in tools_result.tools
        ]
        return available_tools
    ...
# agent.py
from internal_tool import InternalTool
...
    async def run(self):
        try:
            print(
                "Welcome to your AI Assistant. Type 'goodbye' t
            )
            await self.mcp_client.connect()
            available_tools: list[
                InternalTool
            ] = await self.mcp_client.get_available_tools()
            available_tools: list[dict[str, str]] = [
                tool.translate_to_anthropic() for tool in avail
            ]
            await self._refresh()
...
```

In this example, we are working with all three files. First, `internal_tool.py` defines our application’s internal tool representation, which holds translation methods for each model family we want to support. Then, the client’s `get_available_tools()` method is updated to return a list of `InternalTool` objects, offloading the decision of which model to use back to the host application, where it belongs.

NOTE

In the previous example, we wrote a function to translate a Tool object from our internal representation to OpenAI’s function-calling format. OpenAI’s API also supports directly calling remote MCP servers.

Using Multiple Servers

All of the examples so far have shown your client connecting to a single server. And while a single client can only connect to a single server, the Python SDK provides a `ClientSessionGroup` class to help manage multiple sessions concurrently within a client. It does this by connecting either to an existing `ClientSession` with `connect_with_session()` or to a new server by creating a new `ClientSession` with `conenct_to_server()` and then loading all primitives/components (tools, resources, and prompts) offered by the connected server into object properties. Sessions are also stored in the `ClientSessionGroup` object, along with a mapping of each tool to the appropriate session. Sessions themselves can be accessed via the `sessions` property, while the MCP primitives can be accessed via the `prompts`, `resources`, and `tools` properties. Tools in a SessionGroup can still be called with `call_tool()`, just like you would call them in a seingle `ClientSession`. You are able to disconnect from individual servers with `disconnect_from_server()`, which will also remove any loaded server components. In the example below, we will create a simple client that connects to servers via a `ClientSessionGroup`.

Example 4-7. A client that connects to multiple servers via a `ClientSessionGroup`.

```
# client.py
from mcp.client.session_group import ClientSessionGroup, Server

class MCPClient:
    def __init__(
        self,
        name: str,
        llm_client: Anthropic,
    ) -> None:
        self.name = name
        self._llm_client = llm_client
        self._session_group = ClientSessionGroup()
    ...
    async def connect(self, server_parameters: ServerParameters
        """
        Connect to the server set in the constructor.
        """
        connected_server = await self._session_group.connect_to
            server_params=server_parameters,
        )
        connected_server._logging_callback = self._handle_logs
        connected_server._sampling_callback = self._handle_samp
        connected_server._list_roots_callback = self._handle_ro
        connected_server._elicitation_callback = self._handle_e

    async def get_available_resources(self) -> list[Resource]:
        if not self._session_group.sessions:
            raise RuntimeError("Client not connected to a serve

        resources_result = list(self._session_group.resources.v
        if not resources_result:
            logger.warning("No resources found on server")
        return resources_result

    async def get_resource(
        self, uri: str
    ) -> list[BlobResourceContents | TextResourceContents]:
        if not self._connected:
            raise RuntimeError("Client not connected to a serve
        # Read resource from session group
        resource_read_result = await self._session_group.read_r

        if not resource_read_result.contents:
            logger.warning(f"No content read for resource URI {
        return resource_read_result.contents
    ...
    async def disconnect(self) -> None:
        """
        Clean up any resources
        """
        for session in self._session_group.sessions:
            await self._session_group.disconnect_from_server(se

...
# agent.py
async def main():
    """Main async function to run the agent with proper connect
    calculator_server_parameters = StdioServerParameters(
```

```
        command="uv",
        args=[
            "--directory",
            str(Path(__file__).parent.parent.resolve()),
            "run",
            "calculator_server.py",
        ],
    )
    mcp_client = MCPClient(
        name="calculator_multi_client",
        llm_client=anthropic_client,
    )
    await mcp_client.connect(calculator_server_parameters)
    agent = Agent(mcp_client, anthropic_client)
    await agent.run()

if __name__ == "__main__":
    asyncio.run(main())
```

This is simpler from the client perspective, since the `ClientSessionGroup` handles all of the connection management that the single-server client was in previous examples. Our client no longer has as many parameters for its constructor, since it's no longer defined by the parameters of a single server. It also has fewer properties overall: instead of a `_session` property that is `None` until a connection is made, we instantiate a `ClientSessionGroup` object and store it in `_session_group`. All of the methods that call the underlying session's methods (such as `call_tool()`) have to be updated to use `_session_group` instead of `_session`. Additionally, the client no longer has the `_connected` property because the check is no longer necessary: under the hood, the `ClientSessionGroup` treats all the primitives as coming from a single server, and so connecting to the same server multiple times will result in an exception. The full code has removed the checks of the `_connected` property that ensure that the client is connected to a server, and replaced it with a check of the `_session_group.sessions` list to ensure there is at least one connected server.

To get the available tools, resources, and prompts, we have to slightly change the `get_available_*()` methods. These should follow the pattern shown for `get_available_resources()`, where we iterate over the `resources` dictionary in the `ClientSessionGroup` and return the values as a list.

WARNING

Because primitives are treated as coming from a single server, you cannot have primitives with the same name in your `ClientSessionGroup`. The primitive names are stored in a dictionary, and so adding a primitive such as a tool with the same name as an existing one will result in an exception.

Then, we have the `connect()` method, which just takes a `ServerParameters` object. This is actually a Python type alias for `StdioServerParameters`, `SseServerParameters`, or `StreamableHttpParameters`. We use that to connect to a single server, and all connection management is handled by the session group. The flipside to this is that there isn't support yet for including callbacks when establishing the single `ClientSession` objects. A workaround for this is shown in the example: since the `ClientSessionGroup`'s `connect_to_server()` function returns the connected session, you can set the `_logging_callback` and `_sampling_callback` properties on the session directly. This isn't the cleanest way to do this, since the properties are technically private, but until the `ClientSessionGroup` class natively supports callbacks, this is the simplest way to get around this particular limitation. We also updated our `disconnect()` method to iterate over the `sessions` list in the `ClientSessionGroup` and call `disconnect_from_server()` on each one.

The host application also needed to be updated. In the example above, the `if __name__ == "__main__"` block code is largely moved to `main()`. The parameters for our calculator server are stored in a `` StdioServerParameters object, then the `mcp_client `` is instantiated without those parameters. After that, though, we call `connect()` on our own, passing the built server parameters object to the `connect()` method, which creates a session between the client and the server and adds it to the internal list of active sessions.

The `ClientSessionGroup` constructor also takes an optional hook in the parameter `component_name_hook`, which is for renaming a component so as to prevent naming collisions. This is just any function that takes a string (the component name) and an Implementation object, and returns another string (the renamed component).

Best Practices

Some best practices aren't yet fully established, some are established by the community, some are general engineering best practices, but an overview of all of these as they apply to clients will help you quickly build resilient, production-ready MCP clients for your own applications. These revolve around security, connection management, and user experience.

Security

LLMs are powerful tools, but because of their broad generalizability, they have a broad attack surface. Interacting with third-party servers only adds to that. There are a few things you can do to mitigate security risks that come with using the model context protocol. For example, many servers require authentication. For stdio connections, this is best done by accessing credentials from the environment. For streaming HTTP connections, your client should use a Python OAuth2.1 client library to handle the authentication process. The client should initiate the authentication process after accessing the server, which will get the access token from the server to be able to access protected tools and resources.

Resuming Connections

Connections between clients and servers are not always stable. Failure modes abound, and are typically specific to a particular use case. Luckily, streamable HTTP connections are resumable, which allows for broken connections to re-establish themselves where they left off. MCP servers can optionally attach an `id` to each SSE event that they emit. The client can then use this `id` to resume the connection from the last event it received. At the protocol level, this is done by the server including the `Last-Event-ID` header to a GET request to the server it was connected to. This is done similarly in Python.

Your client should be maintaining its session ID and most recent event ID in a property. Before reconnecting, you should take the session and event IDs and add them to a headers dictionary that you will pass to the connection function.

If you remember the streamable HTTP connection section, you might remember that connecting to a server is simpler than connecting to a stdio server. This is also the case for resuming connections. Before resuming, you would add the session and event IDs to the headers dictionary and pass it to the connection function in the headers parameter. In the below example, we simply get the already-saved session ID and event ID, then add them to the headers dictionary. That dictionary is then passed to the streamable HTTP connection constructor.

Example 4-8. Resuming a streamable HTTP connection.

```
from mcp.client.streamable_http import streamablehttp_client


class MCPClient:
    """MCP Client class for connecting to and interacting with

    def __init__(self, name: str, server_url: str) -> None:
        """Initialize the MCPClient with server connection para
        self.name = name
        self.server_url = server_url
        self._session: ClientSession = None
        self.exit_stack = AsyncExitStack()
        self._connected: bool = False
        self._get_session_id: Callable[[], str] = None
        self._last_event_id: str = None

    async def connect(self, headers: dict | None = None) -> Non
        """Connect to the server set in the constructor."""
        headers["Last-Event-ID"] = self._last_event_id

        # Connect to Streamable HTTP server
        streamable_connection = await self._exit_stack.enter_as
            streamablehttp_client(url=self.server_url, headers=
        )
        ...
```

This is a very simpified implementation: we add `_last_event_id` to the `headers` dictionary with the key `Last-Event-ID`, then pass the entire dictionary to the `streamablehttp_client,` with the `headers` parameter.

Paginating Results

When you retrieve results from a server, you may want to paginate them if they’re very long. This is worth following since you don’t know which kinds of agents your users will be using, and you don’t want to overwhelm them with too many results at once. This is typically initiated by the server returning a result with a `nextCursor` property.

If you wish to get the next page of results, you call the same method you used to get the results, but pass in the `nextCursor` property as the `cursor` parameter. This should get you the next page of results from the server. Currently, only the list functions support pagination, in the Python SDK these are represented by `list_resources()`, `list_resource_templates()`, `list_prompts()`, and `list_tools()` methods.

Table of contents collapsed